Nirma University
Institute of Technology



Course Code :- 6CS102CC26
Course Name :- Data Structure

Innovative Assignment 2

Student Name
Vinay Thakor

Student Roll No.
26MCA061

Faculty In Charge
Prof. Nirav Patel

HUFFMAN CODING

An Interactive Encoding, Decoding and Visualisation Tool for Huffman Codes

Innovative Assignment Report

1. Introduction

Huffman coding is a lossless data-compression scheme that's giving every symbol of a text its own variable-length binary code, in many respects; indeed, symbols that are arising often receive short codes and symbols that occur rarely receive long ones, so the encoded text is shorter than a fixed-length representation such as 8-bit ASCII, in real-world scenarios. The codes come from a binary tree built from the symbol frequencies, such as the two lightest nodes are repeatedly merged, under a new parent until a single root remains, and the path from the root to a leaf (left edge = 0, right edge = 1) is the code of that symbol. As, symbols sit only on the leaves, no code is a prefix of another, so the bit stream can, in practice, be decoded without any separators.

The project is implementing the algorithm as an interactive visualisation called Huffman Coding, written in plain HTML, CSS (Tailwind) and JavaScript with SVG graphics and no other libraries, in actual application; and thus, the user types a text of up to 64 characters (with at most 16 distinct symbols) or picks one of six built-in test cases and presses Visualize, broadly speaking. The tool then animates every stage of the algorithm (counting, sorting, each merge, code assignment and encoding) and reports the code table, the encoded bit stream and the compression achieved; hovering a step, a short explanation is shown by a node or a code row, and a decoding section replays the bit stream bit by bit on the tree.

Six test cases were employed, from a single repeated letter to a 36-character sentence with 16 diverse symbols, to a significant degree. What's more, the outputs are reported in Section 6. Every intermediate value is shown in a step table so that the finding can (to a meaningful extent) be confirmed by hand, on the whole.

2. Input Representation and Data Structures

The data are being listed by table 1 that the program works with; let n be the number of characters in the input text and k the number of distinct characters, broadly speaking.

Table 1, such as Notation used by the Huffman method

Name	Type / Size	Meaning
text	string, n	The input to be encoded, including typed by the user or taken from a preset, in actual application.
freq	Map, k entries	freq[c] is the number of times character c emerges; entries are keeping the order of first appearance, broadly speaking.
Node	object	Has an id and a weight w. A leaf also has a character c. An internal node has a left child l and a right child r, in actual application.

Name	Type / Size	Meaning
forest	array of Node	The current list of tree roots, kept sorted by weight; so, it plays the part of the priority queue, broadly speaking.
steps	array	One snapshot of the forest for every stage (Count, Sort, each merge, Codes, Encode), in real-world scenarios.
codes	object, k entries	codes[c] is, in effect, the bit string handed out to character c.
paths	object, k entries	Ids of the nodes from the root to the leaf of c; employed to show a code on hover, broadly speaking.

The characters are being counted by the function build(t), creates the leaves, runs the merge loop, walks the finished tree to generate codes and paths, and returns the list of phases, in actual application; in fact, the function layout(forest) poses from left to right, places each parent at the midpoint of its children and obtains the y position from the depth, in real-world scenarios. The function go(i) is switching to step i, and a requestAnimationFrame loop moves every node towards its new position with rapid smoothing, so merges and re-sorting appear as continuous motion, and the decoder has its own small state (bit list, position, current node, output) recapped in Section 5.1.

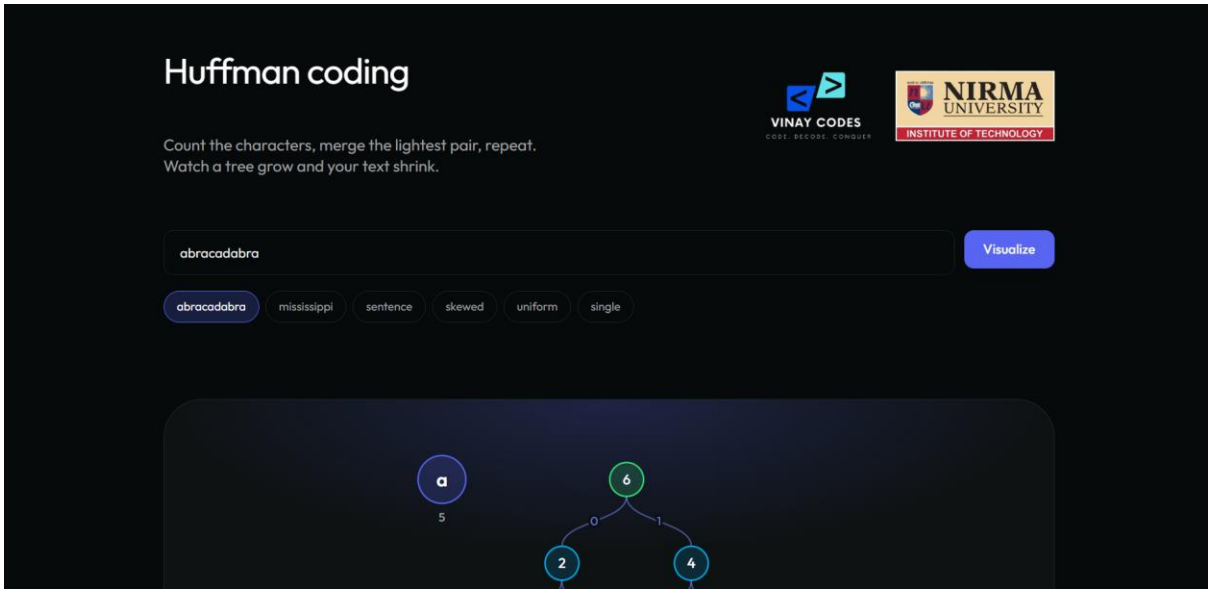


Figure 1, Main window of the Huffman Coding tool showing the title, text input, test-case chips and the Visualize button

3. Huffman Coding Algorithm

The following stages are ran by build(t); the order in which equal weights are handled is fixed so that the same input always is giving the same tree, on the whole; indeed, the standard greedy construction is being followed by this procedure.

- **Initialise.** The leaves are, of course, sorted by weight, ties being broken by first appearance, and form the initial forest, broadly speaking. Each character becomes a leaf whose weight is currently its frequency; count how often every character arises.

- **Merge.** Even as more than one tree is held by the forest, remove the two lightest trees a and b, create a parent of weight a.w + b.w with a as the left child and b as the right child, and insert it before the first tree that's rigorously heavier (that's, after trees of equal weight), broadly speaking. Each merge is recorded as one step.
- **Assign codes.** Walk from the root to every leaf, appending 0 for a left edge and 1 for a right edge, broadly speaking; and thus, the resulting bit string is the code of the leaf. If the text has only one separate character probably, the tree is a single leaf, in real-world scenarios, and it receives the code 0, so every character still costs one bit.
- **Encode.** The size of the finding is the sum of freq[c] x length(codes[c]) over all characters, which is mainly compared with 8n bits, to a significant degree; replace every character of the text by its code and join the codes.

With a binary heap the algorithm is needing O(n + k log k) time: O(n) for counting and O(k log k) for the k, 1 merges, broadly speaking. The visualiser keeps the forest as a sorted array and inserts each new parent in place, which costs O(k) per merge and O(k^2) overall; since k is restricted to 16 this is tiny and keeps every step easy to show, broadly speaking, and encoding and decoding take O(n) time in the length of the text.

The greedy choice is correct, because the two least frequent symbols can always be placed as the deepest sibling leaves of an best tree, including giving a rarer symbol a shorter code than a more frequent one never lowers the total cost, on the whole; indeed, merging them into one node whose weight is their sum decreases the issue to k, 1 symbols, and repeating the input generates a prefix code of minimum average length among all symbol-by-symbol prefix codes, in many respects. How about this, the tie-breaking rule only decides which of several equally good trees is typically drawn; the total number of bits is the same for all of them, in real-world scenarios.

4. Visualising and Explaining the Result

The tool doesn't only print the final codes; every stage can, in practice, maybe be watched, replayed and inspected, on the whole.

- **Step bar.** One button is shown for every stage, namely Count, Sort, one per merge (labelled with the two weights that are joined), Codes and Encode, on the whole; hovering a button displays a one-line explanation with the actual numbers, to take one instance which two weights are joined into the which new weight, on the whole. What's more, clicking it jumps to that stage.
- **Animated tree.** Nodes slide smoothly to their new positions whenever the stage is changing; in turn, the two merged nodes are recapped in cyan and the new parent in green, in many respects. Leaves are showing the character and its weight, edges carry the labels 0, and also, 1, and after the Codes stage each leaf shows its code, on the whole.
- **Hover details.** Hovering a row of the code table shows the path from the root to that leaf in green, in real-world scenarios; and thus, what hovering a node shows is its weight (and its children) or the character count and code, on the whole.

- **Playback.** Play or pause and previous buttons, three speeds and keyboard control (arrow keys and space) are, more accurately, available, in real-world scenarios.
- **Statistics.** After the Encode stage the bit stream is shown with every character in its own colour and can (in actual operation) be copied, in real-world scenarios. As, well, the tool reports the saved percentage, the average number of bits per character and the entropy. 8n bits with the Huffman size are being compared by a bar chart.

The entropy H = - sum p© log2 p©, where p© = freq[c] / n, is the theoretical lower bound for symbol-by-symbol coding, to a significant degree. For a Huffman code the average length L is meeting H <= L < H + 1, which can be inspected in every test case of Section 6.

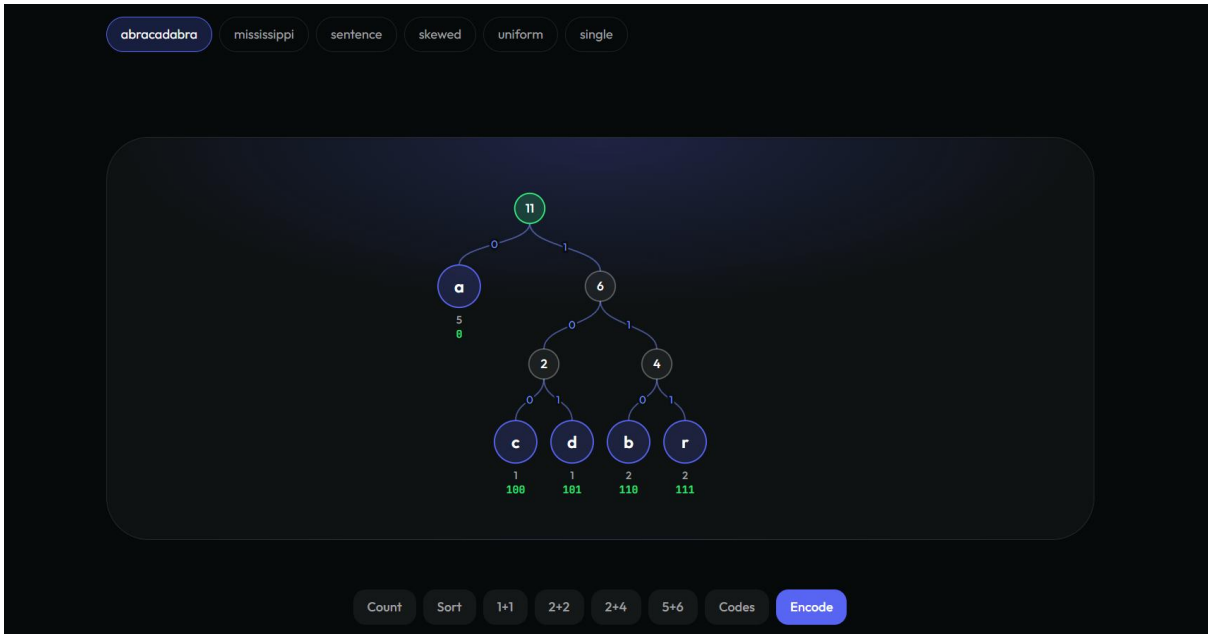


Figure 2 - Tree during construction, namely two merged nodes showed, edge labels 0 and 1 visible

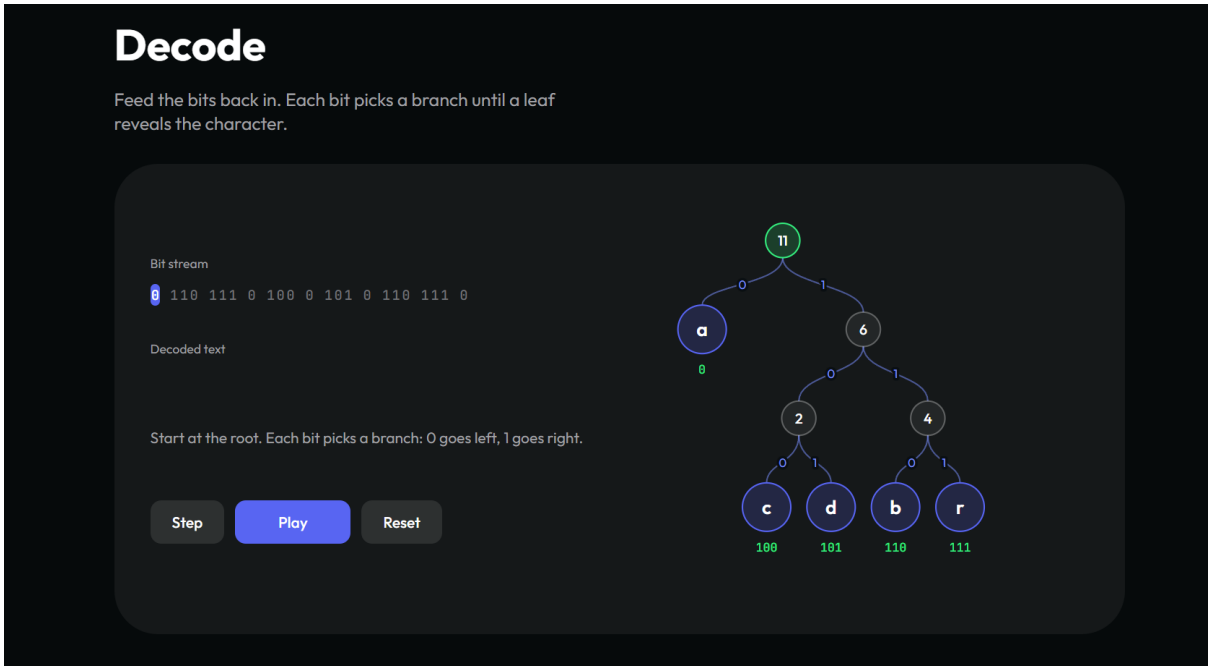
5. Extra Features

5.1 Decoding Visualisation

The Decode section is feeding the encoded bits back into the tree. If the previous step ended on a leaf, the walk first returns to the root, to a significant degree. The function decInit() builds the bit list from the text and its codes, and dstep() consumes one bit per call, in real-world scenarios. A 0 moves to the left child and a 1 to the right child; when the new node is a leaf its character is normally appended to the output. A short message is explaining every step ("Read 1: go right to a node of weight 6", "Reached a leaf: write it down and return to the root"). The bit stream is marking the current bit and colours the bits already read, a small static copy of the tree shows the path taken in green. What's more, at the end the tool is validating that the decoded text equals the original. Step, Play and Reset controls are offered. Table 2 is showing the walk for the text abracadabra.

Table 2, including Decoding walk for abracadabra (Test Case 1)

Symbol	Bits read	Path from the root	Output
1	0	Left	a
2	110	So Right, Left	b
3	111	Fair enough, Right, Right	r
4	0	Left	a
5	100	Okay, Left, Left	c
6	0	Left	a
7	101	Right, Left, Right	d
8	0	Left	a
9	110	Okay, Right, Left	b
10	111	Right, Right, Right	r
11	0	Left	a



During playback: figure 3 - Decode section bit stream, decoded text so far and the showed path on the tree

5.2 Test Cases and Custom Input

Six preset texts are offered as chips, namely abracadabra, mississippi, a sentence, a slanted text, a even text and a single repeated letter, on the whole; picking a chip only fills the input box. Nothing is drawn, until Visualize is pressed. Any text can also be typed by the user; after a successful run the page scrolls to the tree and starts the animation, on the whole. The input is checked, namely an empty text and a text with more than 16 diverse characters are rejected with a message so that the tree stays readable.

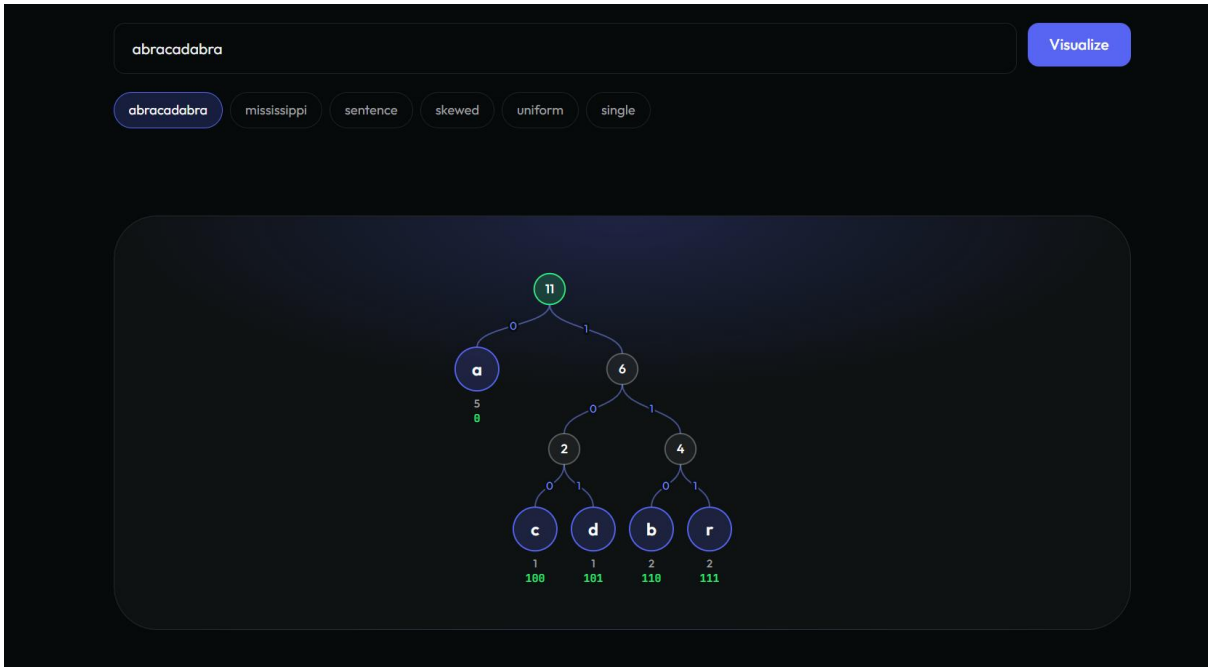


Figure 4 - Input area with the six test-case chips and a custom text entered

5.3 Navigation and Usability

A sticky bar links to the Visualizer, Codes, Decode and Learn sections and shows the section that's currently on screen, on the whole; a short "How it works" section summarises merging, edge labelling, prefix-free codes and the running time, in actual application. The layout adapts to phones and tablets, tooltips follow the pointer without leaving the screen; and thus, what's more, instant transitions instead of animations are gotten by users who prefer lowered motion.

6. Test Cases and Results

Six texts were encoded. Three of them are pored over in detail below. As, well, the remaining three are, in effect, summarised in Section 6.4. The size of the stored tree or code table isn't included in these figures, on the whole. Each time the Huffman size is generally checked with 8n bits perhaps, the size of the same text in 8-bit ASCII, broadly speaking.

6.1 Test Case 1 - Small Text (abracadabra)

The text has 11 characters and 5 separate symbols with frequencies a = 5, b = 2, r = 2, c = 1 and d = 1, broadly speaking; indeed, Table 3 is, in practice, showing the resulting codes and Table 4 the merge steps.

Table 3 - Frequencies and codes of Test Case 1

Character	Frequency	Code	Code length	Bits used
a	5	0	1	5
b	2	110	3	6
r	2	111	3	6
c	1	100	3	3
d	1	101	3	3

Character	Frequency	Code	Code length	Bits used
Total	**11**	**-**	**-**	**23**

Table 4: Merge steps of Test Case 1

Step	Node 1 (lightest)	Node 2	New parent	Roots left
1	c (1)	d (1)	N1 (2)	4
2	b (2)	r (2)	N2 (4)	3
3	N1 (2)	N2 (4)	N3 (6)	2
4	a (5)	N3 (6)	N4 (11)	1

Result: the encoded text is usually 01101110100010101101110, which is 23 bits instead of 88, a saving of 74%, to a significant degree. The letter a covers 5 of the 11 characters and gets the one-bit code 0. Till later, the rare letters c and d need three bits, so the average length is often 2.09 bits per character against an entropy of 2.04 bits.

6.2 Test Case 2: Skewed Frequencies

The text aaaaaaaabbbbccd has 15 characters - a = 8, b = 4, c = 2 and d = 1, in many respects; in fact, one symbol is dominating, so the tree is a chain in which every merge adds the next heavier symbol, in many respects.

Table 5, namely Frequencies and codes of Test Case 2

Character	Frequency	Code	Code length	Bits used
a	8	1	1	8
b	4	01	2	8
c	2	001	3	6
d	1	000	3	3
Total	**15**	**-**	**-**	**25**

Table 6: Merge steps of Test Case 2

Step	Node 1 (lightest)	Node 2	New parent	Roots left
1	d (1)	c (2)	N1 (3)	3
2	N1 (3)	b (4)	N2 (7)	2
3	N2 (7)	a (8)	N3 (15)	1

Result, including the text is needing 25 bits instead of 120, a saving of 79% likely, the best of the six cases, broadly speaking. As, well, the rarest letter d costs three; the leading letter a costs a single bit; in turn, the average length is normally 1.67 bits per character against an entropy of 1.64 bits, on the whole.

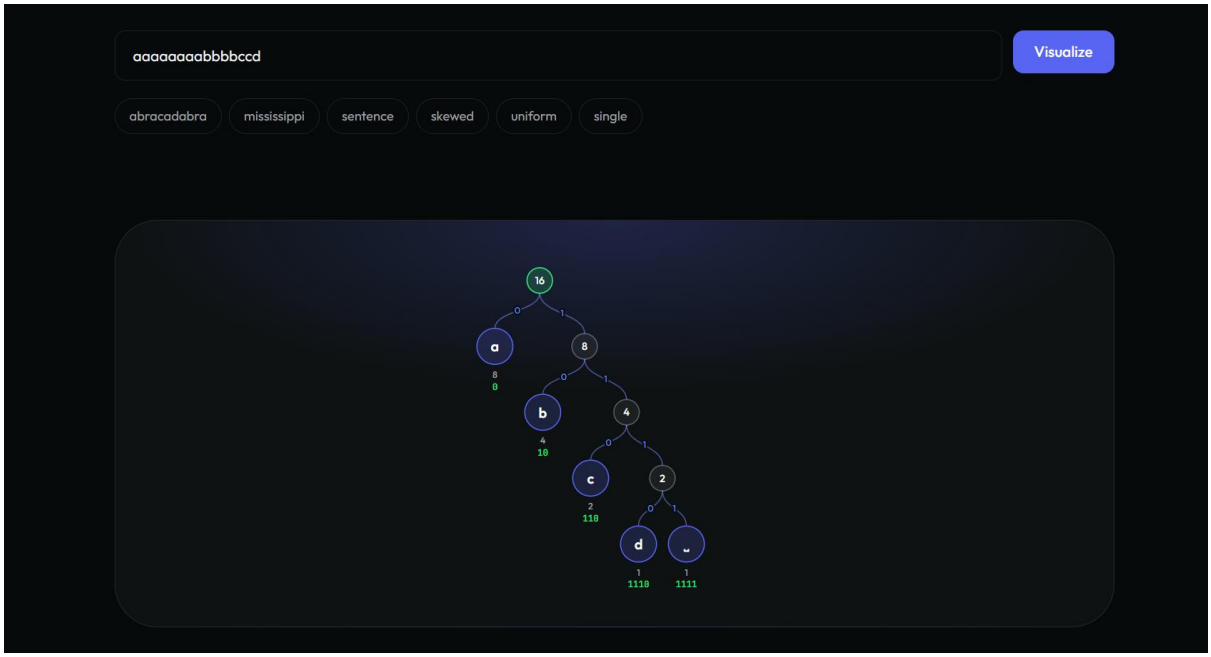


Figure 7, Test Case 2 (slanted text): chain-shaped tree with a one-bit code for the primary symbol

6.3 Test Case 3 - Larger Text with 16 Symbols

The sentence “this is an example of a 10uffman tree” has 36 characters and 16 separate symbols, including the space (7 happenings); it needs 15 merges, so the animation is showing several pairs of equal-weight nodes being joined before the larger subtrees combine, broadly speaking.

Table 7, such as Frequencies and codes of Test Case 3

Character	Frequency	Code	Code length	Bits used
space	7	111	3	21
a	4	000	3	12
e	4	001	3	12
f	3	1101	4	12
t	2	0100	4	8
h	2	0101	4	8
i	2	0110	4	8
s	2	0111	4	8
n	2	1000	4	8
m	2	1001	4	8
x	1	10100	5	5
p	1	10101	5	5
l	1	10110	5	5
o	1	10111	5	5
u	1	11000	5	5
r	1	11001	5	5

Character	Frequency	Code	Code length	Bits used
Total	**36**	**-**	**-**	**135**

Table 8: Merge phases of Test Case 3

Step	Node 1 (lightest)	Node 2	New parent	Roots left
1	x (1)	p (1)	N1 (2)	15
2	l (1)	o (1)	N2 (2)	14
3	u (1)	r (1)	N3 (2)	13
4	t (2)	h (2)	N4 (4)	12
5	I (2)	s (2)	N5 (4)	11
6	n (2)	m (2)	N6 (4)	10
7	N1 (2)	N2 (2)	N7 (4)	9
8	N3 (2)	f (3)	N8 (5)	8
9	a (4)	e (4)	N9 (8)	7
10	N4 (4)	N5 (4)	N10 (8)	6
11	N6 (4)	N7 (4)	N11 (8)	5
12	N8 (5)	space (7)	N12 (12)	4
13	N9 (8)	N10 (8)	N13 (16)	3
14	N11 (8)	N12 (12)	N14 (20)	2
15	N13 (16)	N14 (20)	N15 (36)	1

Result - the sentence needs 135 bits instead of 288, a saving of 53%; indeed, the saving is smaller than in Test Cases 1 and 2, because the frequencies are spread over many symbols, to a significant degree. The most frequent symbol (space) has a three-bit code, the symbols with frequency 1 code, and the average length is 3.75 bits per character against an entropy of 3.71 bits, on the whole.

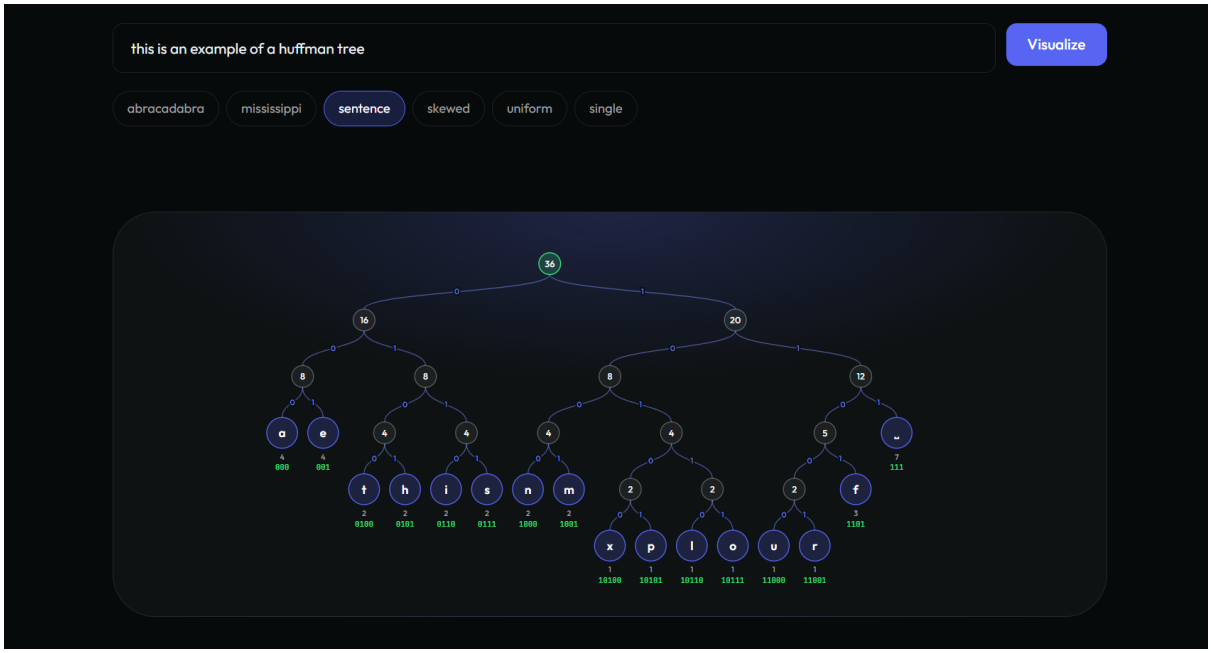


Figure 6 - Test Case 3 (sentence): tree with 16 leaves and the step bar with 15 merges

6.4 Further Test Cases

The remaining three presets test standard and degenerate inputs; in turn, they're using the same method and are covered in the comparison in Section 7, in many respects.

Table 9, such as Codes of the remaining test cases

Text	Codes	Bits (Huffman / ASCII)
mississippi	s = 0, i = 11, p = 101, m = 100	21 / 88
abcdefgh	a = 000, b = 001, c = 010, d = 011, e = 100, f = 101, g = 110, h = 111	24 / 64
zzzzzz	z = 0	6 / 48

For mississippi the two most widespread letters s. Also, i get the shortest codes and the saving is typically 76%; that said, one bit per character is still needed by the code although the entropy is usually zero, in actual application, and every code has three bits, so Huffman coding degenerates to a fixed-length code (saving 63% only, because ASCII is employing 8 bits), on the whole. For abcdefgh every frequency is generally 1, the tree becomes perfectly balanced, and for zzzzzz the tree is a single leaf with the code 0; the saving of 88% is, by design, real against ASCII.

7. Comparison and Discussion

Table 10: Summary of the six test cases

Test case	n x k	ASCII bits	Huffman bits	Saved	Avg. bits / char	Entropy
1. abracadabra	11 x 5	88	23	74%	2.09	2.04
2. Skewed	15 x 4	120	25	79%	1.67	1.64

Test case	n x k	ASCII bits	Huffman bits	Saved	Avg. bits / char	Entropy
3. Sentence	36 x 16	288	135	53%	3.75	3.71
4. mississippi	11 x 4	88	21	76%	1.91	1.82
5. Uniform	8 x 8	64	24	63%	3.00	3.00
6. Single	6 x 1	48	6	88%	1.00	0.00

- **Slanted input compresses best**. Test Case 2 saves 79% with only four symbols; so, the more uneven the frequencies, the shorter the average code, broadly speaking. Till then, test Case 3, whose frequencies are spread over 16 symbols, saves 53%, broadly speaking.
- With eight equally frequent symbols Huffman coding gives the same three bits per character as a plain 3-bit code; the entropy and the average length are often both 3.00, to a significant degree; over fixed-length codes, nothing is being gained by homogeneous input.
- **Huffman stays close to the entropy.** Only the single-symbol case is reaching the full extra bit; in fact, every time with at least two symbols the average length is, in practice, above the entropy and less than one bit above it, as the theory anticipates, in actual application.
- **Overhead isn't counted.** A decoder is needing the tree or the frequency table. For very short texts such as zzzzzz this overhead can be larger than the saving, so the percentages are outlining the encoded data only.
- **Ties differently tree. Even as, the total number of bits would be same; a few nodes often have the same weight, to take one instance the five leaves of weight 1 in Test Case 3, on the whole.** A diverse tree and distinct codes would be drawn by another tie-breaking rule.
- **Explanation matters.** The step table and the animated merges let another person repeat the construction by hand and reach the same codes, which is turning the program from a calculator into a tool for discerning how Huffman coding works, on the whole.

8. Conclusion

Huffman Coding writes the greedy Huffman algorithm for any text of up to 64 characters, builds the tree step by step, designates prefix-free codes, encodes the text and decodes it again on the same tree, to a significant degree. The six test cases, from a single repeated letter to a 36-character sentence, behaved as the theory predicts, like savings between 53% and 88% against 8-bit ASCII, an average code length always within one bit of the entropy, and no saving beyond fixed-length coding for consistent input, to a significant degree. The animated tree, the hover explanations, the colour-coded bit stream and the decoding walk are making the behaviour of the algorithm visible and easy to check, and show how a small change in the frequencies changes the tree and the length of the encoded text.